Tennis Analyzer

Technical design and algorithm development

Lily Wang

Purpose and contribution

I developed this application to help junior tennis players receive automated feedback on their technique. This project was inspired by teaching children through the Austin Great Wall Chinese School tennis program. I wanted to give players a way to continue practicing and improving outside of lessons.

I designed the application's functionality, interface, styling, and analysis algorithms, with programming assistance from Claude Code for advanced functions. My technical work focused on detecting strokes from body motion, developing a motion-based ball detector, and converting measurements into understandable feedback. MediaPipe Pose provides the pretrained body-landmark model, which is processed through JavaScript.

Design

The application uses MediaPipe Pose to estimate 33 body landmarks and their visibility in each processed frame. Wrist motion and body measurements are calculated to detect strokes, assign stroke types, and generate scores and feedback. A separate algorithm searches for possible ball locations through pixel changes. The interface displays the results through pose overlays, timeline markers, and a feedback panel.

Component	Application Role
MediaPipe Pose	Pretrained body-landmark estimation
Custom JavaScript	Stroke detection, classification, scoring, and ball-motion analysis
TensorFlow.js COCO-SSD	Pretrained object detection for the racquet overlay
Firebase	Authentication, uploaded videos, and saved analysis results

A background scan samples video frames at 0.2-second intervals and stores detected stroke results. Live playback processes pose results available on the device using slightly adjusted detection thresholds. To avoid duplicate strokes returned by the different scans, a recent live detection can update a stored stroke's type and timing while keeping the displayed score consistent. The following sections describe current functionality.

User input and interface

Video library and saved analysis

Firebase provides authentication, video storage, and stored analysis data. Each authenticated user has a separate video library. The original video file is stored in Firebase Storage, while Firestore stores information such as the video's name, file size, upload time, storage path, and download URL. The library can be opened as a sidebar and displays videos in upload order, allowing videos to be selected, renamed, or permanently deleted (from Firestore as well).

Background scan results are also saved with each video. Stored information includes detected stroke start/end times, classification, score, measurements, feedback, reliability warnings, and timestamps of false stroke detections (marked by the user). A scan version number identifies results produced by the current algorithm. When a video is opened again, complete results from the current version can be loaded immediately instead of rescanning the entire video.

Recording and live analysis

The recording feature allows the application to analyze a player directly from the device camera instead of requiring an existing video. The browser's getUserMedia API requests camera and microphone access. The user can switch between front and rear cameras and pinch-to-zoom (when available), which scales the video preview and pose overlay together.

While the camera is open, frames are continuously sent to MediaPipe Pose. The recording feedback panel appears beside the preview window and provides live feedback from calculated joint angle measurements. If few body landmarks are visible, the panel asks the player to move more fully into frame. Because a full stroke might not be completed when the panel updates, it doesn’t display a stroke classification or score. Stroke detection continues running while recording. After a stroke finishes, optional voice feedback reads the most severe coaching feedback generated.

The video itself is recorded using the browser's MediaRecorder API and a video format is selected depending on the browser. After stopping the recording, the user can name it and *Upload & Analyze*, saving it to the user’s video library. The separate background scan is skipped for clips recorded inside the application, they’re analyzed through live detection during playback instead.

Playback and review features

Several controls were added to make the analyzer useful for reviewing practice footage.

Mark Play Area: The user can draw a polygon around the relevant portion of the court by placing at least three points on the video. During live ball detection, candidate pixels outside this polygon are ignored, reducing false detections from spectators, neighboring courts, or other movement. This feature works best with a stationary camera.

Auto-pause: When enabled, playback pauses shortly after each stored stroke to give the user time to read the score, classification, measurements, and feedback. The user can then continue to the next stroke. Without auto-pause, playback is continuous and feedback updates as the video reaches each detected stroke.

Stroke markers and correction: Detected strokes appear as clickable markers along the video progress bar. If a detection is not actually a tennis stroke, the user can select *Not a stroke* to remove it. Its timestamp is saved so that live detection does not immediately recreate the same marker. An undo option restores an accidentally removed stroke. The user can also clear stored results and rescan the full video.

Visibility and reliability warnings: If body landmarks disappear for an extended period, a warning appears that the player is outside the frame. Stored strokes with low body visibility display a similar warning of unreliability. The user can skip directly to the next stroke with sufficient body visibility.

Other controls: The settings menu allows the user to select right- or left-handed play and toggle ball and joint-angle overlays, voice feedback, desktop/mobile layouts, and fullscreen mode. Handedness is used by the analysis when selecting the dominant elbow and assigning forehand/backhand qualifiers. Video review supports playback speeds from 0.25× to 2× along with forward and backward skips.

Detecting and classifying strokes

Measuring wrist movement

For each usable pose, the more visible wrist is selected and its displacement is divided by elapsed video time to find speed. This speed is normalized using shoulder width to reduce the effect of different camera zooms. The starting threshold for the background scan is lower because wrist speed can peak quickly during a stroke, and the periodic sampling will likely miss the highest speed frames. The following thresholds provided the most reliable results after testing:

Setting	Live detection	Background scan
Sampling	Available pose results	Every 0.2 seconds
Start speed threshold	0.65	0.25
End speed threshold	0.25	0.20
Consecutive samples needed	3	1

Recovering preparation and confirming stroke finish

A rolling two-second wrist history allows slower preparation time to be added before the wrist acceleration in the middle of the stroke. When the thresholds are reached, the algorithm searches the history for the wrist position closest to the horizontal shoulder midpoint (taking into account the previous stroke's cooldown period). That timestamp becomes the stroke's start and is marked on the video progress bar.

Separate start/end thresholds help prevent brief speed fluctuations from forming multiple strokes. The detector checks for wrist slowing over 0.35 seconds through displacement, sample-count conditions, or a timeout after the last fast movement. A cooldown period delays the next stroke detection.

Assigning stroke types

Strokes are classified using wrist height and trajectory. For example, a wrist above the shoulder level can indicate a serve or overhead. Handedness and body-relative wrist movement provide forehand or backhand qualifiers for slices and volleys. Ordinary strokes currently use the single label *Groundstroke*.

I added a delayed serve check because a pause near the trophy position could end detection before the actual ball contact. Volley classification runs only during live playback due to unreliable detection in the scan.

Measurements, scoring, and feedback

I designed the feedback around measurements that I could calculate from pose landmarks and connect to tennis technique. Elbow and knee angles, shoulder-line tilt, and trunk lean are computed and averaged from normalized image coordinates. The ranges below define scoring rules based on standard technique:

Measurement	Range receiving full credit	Weight
Mean knee angle	115–152° for other strokes 100–178° for Serve / Overhead	30
Selected elbow angle	100–162°	30
Shoulder-line tilt	0–7°	20
Trunk lean	0–11°	20

For scoring, a measurement earns its full weight inside the stated range. Outside that range, it earns 55% between 85% * lower bound and 110% * upper bound, and 12% otherwise. Earned points are divided by total weight and rescaled to the nearest integer score out of 100.

Handling uncertain measurements

Incorrectly detected landmarks sometimes produce unrealistic angles resulting in very low scores. I added checks that give unusually far out-of-range measurements one-tenth of their normal weight. Missing measurements (usually from hidden landmarks) contribute neither points nor weight. A separate check for unusual vertical ordering of body landmarks produces a warning that body detection may be unreliable.

However, an incorrect pose can still produce plausible angles. Camera view and averaging across a movement limit how precisely these measurements can describe technique.

Converting measurements into feedback

The stroke score and coaching feedback are generated separately. The score combines the available measurements into a weighted total, while the feedback system converts individual measurements into simple coaching messages. Feedback cards are labeled as positive feedback, improvement warnings, informational observations, or tips.

For completed strokes, the feedback is generated from measurements averaged across the detected stroke. During the live recording view, the same general feedback rules are applied to the current pose instead so that the player can see immediate changes in body position. There is a prewritten set of feedback for each angle measurement, with differing content depending on whether the measurement is inside or outside the range. For example, for the dominant elbow angle:

Measurement	Example condition	Feedback
Dominant elbow angle	100–162°	Good arm extension
	Above 162°	Add slight elbow bend
	Below 65°	Extend arm at contact

Racquet detection can add an informational card when COCO-SSD identifies the racquet in the frame. Values outside the plausibility limits are not treated as normal technique errors because they are more likely to come from incorrect landmark placement. Missing measurements are shown as low visibility, while implausible values are marked as possibly inaccurate.

Scoring and feedback example

Measurement	Value	Earned points	Feedback
Knee angle	160°	30 × 0.55 = 16.5	Bend knees a bit more
Elbow angle	120°	30	Good arm extension

Shoulder-line tilt	6°	20	Shoulders level
Trunk lean	10°	20	Good upright posture
		Total: 86.5 out of 100	

The knee angle is slightly outside the range so the points are decreased, and the final displayed score is 87. Feedback is separate for each measurement category. The score summarizes the angle measurements but doesn't reflect accuracy or a player rating.

Motion based ball detection

I first tried yellow and green color thresholds to locate the ball. However, unclear footage results in balls being missed in the shadows and false detections on green court surfaces or white lines. I changed my approach to motion, looking for pixels that change color twice within a short interval.

The current detector first estimates a small global translation between frames to compensate for slight camera movement. It compares each pixel with the shifted previous image and marks candidate pixels with two significant color changes within six detector updates. A loose color check eliminates colors that are strongly red or magenta ranges. Nearby candidate pixels are grouped, size/shape filters keep small, compact, ball-shaped clusters, and a proximity check rejects cluster pairs that may be fragments of a larger moving object. The detector returns the center of the largest surviving cluster.

I used debugging overlays showing motion pixels and bounding boxes to inspect false candidates and revise the rules. The final output remains a possible ball location because moving players, shadows, camera changes, and other motion can decrease certainty.

Ball detection acts as a secondary check for stroke detection during the background scan. The scan keeps a short history of recent detected ball positions. Once three usable positions are available, it compares the direction of the two consecutive motion vectors. If the ball has moved far enough in both intervals with a strong change in direction, the event can trigger the beginning of a possible stroke.

Development testing and next steps

I tested the analyzer with self-recorded GoPro match footage and close-up videos of myself performing strokes with different techniques. Match footage exposed difficulties when the player appeared small, moved out of frame, or was partly obscured. Close-up clips made it easier to inspect the landmarks and resulting feedback. Extensive testing and observations informed the chosen landmarks, thresholds, and design choices.

I tested by comparing detected strokes with manually labeled examples and recording missed strokes, extra detections, and timing errors across different camera views. I applied my technique expertise as a competitive player with coaching experience to evaluate the usefulness of the scoring and feedback. Further development could include separating stroke phase (preparation, contact, and follow-through) measurements without averaging, applying stroke-specific feedback, and testing on a wider range of players and videos.

Source notes

GitHub link: https://github.com/gemini6829/tennis_analyzer/tree/main
Last updated October 24, 2026.